

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)**

Факультет компьютерных технологий и прикладной математики

Кафедра информационных технологий

ОТЧЕТ

о выполнении лабораторной работы № 4

по дисциплине «Программная инженерия»

Выполнила: ст. гр. МО 32/2

Переузник В. С.

Проверил: доцент каф. ИТ

Полетайкин А. Н.

Краснодар

2026

Тема: разработка информационного обеспечения ПС.

Цель: изучение программных средств для разработки моделей информационной базы ПС, проработка методов нормализации отношений в БД, приобретение навыков применения CASE-средства ERwin для моделирования базы данных ПС.

Задание:

1. На основе диаграммы классов UML, разработанной при выполнении лабораторной работы 2, произвести идентификацию сущностей информационной базы ПС и связей между ними.
2. При помощи CASE-средства ERWin разработать ER-диаграмму логической модели данных по принципу: один класс – одна сущность.
3. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы 1, провести обоснованный выбор СУБД.
4. Провести нормализацию сущностей логической модели данных и при помощи CASE-средства ERWin разработать ER-диаграмму физической модели данных с учетом выбранной СУБД.
5. Средствами ERWin на основе физической модели данных выполнить генерацию SQL-кода для создания реляционной базы данных ПС.
6. В выбранной СУБД развернуть БД, доработать её структуру с учетом возможной нормализации отношений, а также доработать структуру таблиц с учетом ограничений на значения полей. В отчете представить изображение, подтверждающее факт развертывания БД.
7. Выполнить описание таблиц БД в формате табл. 3 с указанием для всех полей типов данных выбранной СУБД. Столбцы «Условие на значение», «Значение по умолчанию» и «Примечание» заполнять при необходимости. Выполнить краткое описание таблиц БД и связей между ними в формате таблиц.

Ход выполнения работы:

1. Идентификация сущностей информационной базы ПС

На основе диаграммы классов UML произведена идентификация сущностей, информация о которых подлежит хранению в информационной базе программной системы.

На диаграмме классов у меня было описано 9 классов: Менеджер, Парсер, Уравнения, Интегралы, Диффуры, Системы, База Данных, Неросеть и

Визуализация. Однако ни один из представленных классов не является хранилищем данных, все они являются функциональными компонентами системы. Классы Уравнения, Интегралы, Диффуры, Системы являются вычислительными компонентами, которые выполняют расчеты, но не сохраняют результаты. Класс База Данных представлен как компонент доступа, но сама структура сохраняемых данных на диаграмме классов не описана.

В ходе анализа диаграммы классов и требований к хранению данных выделена одна основная сущность, подлежащая хранению в информационной базе - *обучающие данные (TrainingData)*.

Данная сущность предназначена для накопления пар «входные данные - результат» с целью формирования обучающей выборки для нейросетевой подсистемы. Каждая запись представляет собой законченное решение математической задачи, выполненное пользователем (студентом или преподавателем) в процессе работы с системой. Сбор данных происходит автоматически, без явного уведомления пользователя.

На диаграмме классов данная сущность соответствует классам Нейросеть (метод обучиться()), как источник потребления данных), База Данных (метод сохранить()), как механизм хранения), а также результатам работы методов решитьУравнение(), решитьИнтеграл(), решитьДиффур(), решитьСистему() класса Менеджер, как источник данных.

В связи с тем, что информационная база содержит одну основную сущность, связи между сущностями отсутствуют. Данные хранятся в виде независимых записей, каждая из которых представляет собой отдельный пример для обучения нейросети.

2. Разработка ER-диаграммы логической модели данных

На основе идентифицированных сущностей при помощи CASE-средства ERwin разработана ER-диаграмма логической модели данных. Построение выполнено с учетом целевого назначения информационной базы - накопление данных для обучения нейросетевой подсистемы.

Логическая модель содержит одну сущность - TrainingData (Обучающие данные). Данная сущность агрегирует информацию из классов диаграммы UML, подлежащих хранению: Уравнения, Интегралы, Диффуры, Системы, Нейросеть, База Данных. Объединение в одну сущность обусловлено необходимостью формирования единой выборки данных для обучения нейросети, а также

использованием JSONB-формата в целевой СУБД PostgreSQL, позволяющего хранить разнородные параметры задач в структурированном виде.

Атрибуты сущности TrainingData: уникальный идентификатор (id), тип решаемой задачи (task_type), условие задачи и параметры в формате JSON (input_data), название использованного метода (method_used) и результат решения в формате JSON (result).

На рисунке 1 представлена ER-диаграмма логической модели данных, разработанная в ERwin

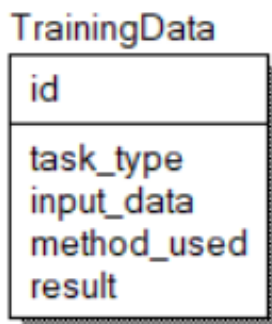


Рисунок 1. ER-диаграмма логической модели данных

3. Выбор СУБД

В соответствии с требованиями технического задания, разработанного в лабораторной работе №1, проведен обоснованный выбор системы управления базами данных. В техническом задании были определены следующие требования к информационному обеспечению:

- 1) Хранение данных осуществляется в формате JSON
- 2) База данных предназначена для накопления пар «входные данные – результат» для обучения нейросети
- 3) Данные имеют слабо структурированную форму, так как параметры различных типов задач (уравнения, интегралы, ОДУ, СЛАУ) имеют разный набор полей

На основе анализа требований технического задания и существующих решений выбрана СУБД PostgreSQL.

Обоснование выбора: PostgreSQL поддерживает тип данных **JSONB**, что позволяет хранить разнородные параметры задач в гибком формате без создания множества таблиц. При этом сохраняются преимущества реляционной СУБД: поддержка ACID-транзакций, надежность хранения, возможность выполнения сложных запросов при необходимости. Выбранная

СУБД распространяется бесплатно и имеет хорошую интеграцию с выбранным стеком технологий (FastAPI, Python).

4. Нормализация и разработка физической модели данных

Сначала был проведен анализ сущности TrainingData на соответствие нормальным формам.

- 1) 1НФ (атомарность). Все атрибуты являются атомарными: id - числовой идентификатор, task_type - строковое значение, input_data - цельный JSON-документ, method_used - строковое значение, result - цельный JSON-документ.
- 2) 2НФ (отсутствие частичных зависимостей). Первичный ключ id является простым (не составным), следовательно, частичные зависимости отсутствуют.
- 3) 3НФ (отсутствие транзитивных зависимостей). Все неключевые атрибуты (task_type, input_data, method_used, result) функционально зависят только от первичного ключа id. Транзитивные зависимости отсутствуют.

Таким образом, сущность TrainingData находится в 3НФ. Дополнительная нормализация не требуется, так как дальнейшее разделение сущности привело бы к неоправданному усложнению структуры хранения данных для обучения нейросети

На основе логической модели данных в CASE-средстве ERwin разработана физическая модель с учетом выбранной СУБД. Однако используемая версия ERwin не содержит встроенной поддержки PostgreSQL, поэтому в качестве целевой СУБД при моделировании выбран универсальный драйвер ODBC/Generic

Спроектированная физическая модель ориентирована на последующее развертывание в СУБД PostgreSQL. Ниже приведено соответствие типов данных, использованных в ERwin, и типов данных PostgreSQL, которые будут применены при генерации SQL-кода и создании базы данных:

Атрибут	Тип в ERwin (ODBC/Generic)	Тип в PostgreSQL	Назначение
id	INTEGER	SERIAL	Уникальный идентификатор (автоинкремент)
task_type	VARCHAR(20)	VARCHAR(20)	Тип решаемой задачи
input_data	LONG VARCHAR	JSONB	Условие задачи в формате JSON

method_used	VARCHAR(50)	VARCHAR(50)	Название численного метода
result	LONG VARCHAR	JSONB	Результат решения в формате JSON

Таблица 1. Соответствие типов данных ERwin и PostgreSQL

На рисунке 2 представлена ER-диаграмма физической модели данных, разработанная в ERwin.

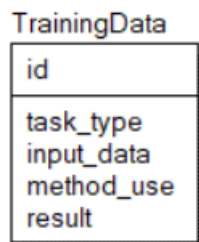


Рисунок 2. ER-диаграмма физической модели данных

5. Генерация SQL-кода средствами ERwin

На основе физической модели данных средствами ERwin выполнена генерация SQL-кода для создания БД. В результате генерации получен следующий SQL-скрипт:

```

CREATE TABLE TrainingData
(
    id INTEGER NOT NULL,
    task_type VARCHAR(20) NOT NULL,
    input_data VARCHAR(20) NOT NULL,
    method_use VARCHAR(20) NULL,
    result VARCHAR(20) NOT NULL
)
;

ALTER TABLE TrainingData
    ADD PRIMARY KEY (id)
;
  
```

Рисунок 3. Генерация SQL-кода

Сгенерированный код является универсальным (ODBC/Generic). Для развертывания в выбранной СУБД PostgreSQL требуется адаптация

типов данных с учетом требований технического задания.
Адаптированный SQL-скрипт для PostgreSQL:

```
CREATE TABLE TrainingData (  
    id SERIAL PRIMARY KEY,  
    task_type VARCHAR(20) NOT NULL,  
    input_data JSONB NOT NULL,  
    method_used VARCHAR(50),  
    result JSONB NOT NULL  
);
```

6. Развертывание базы данных

В соответствии с выбранной СУБД PostgreSQL выполнено развертывание базы данных для программной системы.

Для автоматического развертывания базы данных разработан скрипт `init_db.py`, который выполняет следующие операции:

- 1) Создание базы данных `math_solver_db` (если она не существует)
- 2) Создание таблицы `training_data` с необходимыми полями
- 3) Добавление ограничения на допустимые значения поля `task_type`

```
init_db.py U X
init_db.py
1 import psycopg2
2 from psycopg2 import sql
3
4 conn = psycopg2.connect(
5     host="localhost",
6     database="postgres",
7     user="postgres",
8     password="12345"
9 )
10 conn.autocommit = True
11 cur = conn.cursor()
12
13 #создаем бд
14 cur.execute("SELECT 1 FROM pg_database WHERE datname='math_solver_db'")
15 if not cur.fetchone():
16     cur.execute(sql.SQL("CREATE DATABASE {}").format(sql.Identifier("math_solver_db")))
17     print("бд создана")
18 else:
19     print("бд уже существует")
20
21 #подключаемся к бд
22 conn.close()
23 conn = psycopg2.connect(
24     host="localhost",
25     database="math_solver_db",
26     user="postgres",
27     password="12345"
28 )
29 cur = conn.cursor()
30
31 #создаем таблицу
32 cur.execute("""
33 CREATE TABLE IF NOT EXISTS training_data (
34     id SERIAL PRIMARY KEY,
35     task_type VARCHAR(20) NOT NULL,
36     input_data JSONB NOT NULL,
37     method_used VARCHAR(50),
38     result JSONB NOT NULL
39 )
40 """)
41 print("таблица создана")
42
43 #добавляем ограничение для task_type
44 try:
45     cur.execute("""
46 ALTER TABLE training_data
47 ADD CONSTRAINT check_task_type
48 CHECK (task_type IN ('equation', 'integral', 'ode', 'slae'))
49 """)
50     print("ограничение добавлено")
51 except psycopg2.errors.DuplicateObject:
52     print("ограничение уже существует")
53
54 cur.close()
55 conn.close()
```

Рисунок 4. Код выполнения

В результате выполнения скрипта база данных `math_solver_db` и таблица `training_data` были успешно созданы.

На рисунке 5 представлено изображение терминала, подтверждающее факт успешного развертывания базы данных.

```
PS C:\Users\user\intelligent-math-solver> C:\Users\user\AppData\Local\Programs\Python\Python313\python.exe init_db.py
бд создана
Таблица создана
```

Рисунок 5. Результат развертывания базы данных в терминале VS Code (первый запуск)

```
PS C:\Users\user\intelligent-math-solver> C:\Users\user\AppData\Local\Programs\Python\Python313\python.exe init_db.py
бд уже существует
таблица создана
ограничение добавлено
```

Рисунок 6. развертывания бд с добавленными ограничениями

7. Описание таблиц базы данных

В таблице 2 представлено описание структуры таблицы `training_data` с указанием типов данных PostgreSQL, ограничений и значений по умолчанию.

Имя поля	Тип данных	Размер, байт	Условие на значение
<code>id</code>	SERIAL	4	PRIMARY KEY
<code>task_type</code>	VARCHAR(20)	20	CHECK (task_type IN ('equation', 'integral', 'ode', 'slae'))
<code>input_data</code>	JSONB	переменный	NOT NULL
<code>method_used</code>	VARCHAR(50)	50	-
<code>result</code>	JSONB	переменный	NOT NULL

Таблица 2. Структура таблицы `training_data`

В таблице 3 представлен перечень разработанных таблиц ПС (в моем случае - одна)

№ п/п	Имя таблицы	Описание
1	<code>training_data</code>	Хранит пары «входные данные – результат» для формирования обучающей выборки нейросетевой подсистемы. Каждая запись содержит условие задачи, тип задачи, использованный метод и полученный результат

Таблица 3. Список разработанных таблиц

Моя база данных содержит одну таблицу, поэтому связи между таблицами отсутствуют. Данные хранятся в виде независимых записей, каждая из которых представляет собой отдельный пример для обучения нейросети

Вывод: в ходе выполнения лабораторной работы спроектирована информационная база программной системы для решения математических задач. На основе диаграммы классов UML выделена сущность TrainingData, разработаны логическая и физическая модели данных в ERwin, обоснован выбор СУБД PostgreSQL. Выполнено развертывание базы данных, сгенерирован и адаптирован SQL-код, описана структура таблицы в соответствии с требованиями технического задания.